

**Loong with Blended-block Errors and Augmented Gabidulin Codes
(BAG-Loong)**

Contents

1	Description of BAG-Loong	2
1.1	Overview of BAG-Loong	2
1.2	Notation and Symbol Declaration	2
1.3	IND-CPA Secure BAG-Loong.PKE	4
1.4	IND-CCA2 Secure BAG-Loong.KEM	5
1.5	Parameter Selection	6
2	Design Rationale	8
2.1	Difficult Problems for Code-based Public-key Cryptography	8
2.2	Criterion for Parameter Choices	9
2.3	Augmented Gabidulin Codes and Decoding	10
2.4	Statement of Design	11
3	Security Anaysis	12
3.1	Security Analysis in the Classical Model	12
3.2	Security Analysis in the Quantum Model	14
3.3	Known Attacks	15
3.3.1	OJ Combinatorial Attacks	15
3.3.2	PRR Attacks	17
3.3.3	Algebraic Attacks	20
3.4	Estimates on Concrete Security Strength	21
4	Performance Analysis	21
5	The Features of BAG-Loong	23
5.1	The main Advantages of BAG-Loong	23
5.2	Comparison with Other Unstructured Cryptosystems.	23
	References	24

1 Description of BAG-Loong

1.1 Overview of BAG-Loong

BAG-Loong.KEM is a code-based IND-CCA2 secure key encapsulation Mechanism (KEM for short) with no algebraic structure under rank metric, obtained by Fujisaki–Okamoto (SFO) transform on an IND-CPA secure public key encryption algorithm BAG-Loong.PKE. We provide the parameter sets targeting classical security levels of 128, 256, 384, and 512 bits, while achieving 80, 128, 192, and 256 bits of quantum security, respectively.

1.2 Notation and Symbol Declaration

We introduce the needed notations and concepts used throughout the construction of the BAG-Loong.PKE and BAG-Loong.KEM.

Basic algebraic structures. Let q be a prime power and let $\mathbb{F}_{q^m}$ denote the finite extension field of degree m over $\mathbb{F}_q$. we take $q = 2$. Vectors are denoted by bold lower-case letters, and matrices by upper-case letters. Unless otherwise specified, all vectors are assumed to be row vectors.

Support and rank weight. For any $V = (v_{i,j}) \in \mathcal{R}^{r \times t}$, the support of V is defined as the $\mathbb{F}_q$ -subspace generated by all its entries, i.e., $\text{Supp}(V) = \langle v_{i,j} : 1 \leq i \leq r, 1 \leq j \leq t \rangle_{\mathbb{F}_q}$. The rank weight of V is defined as $w_R(V) = \dim(\text{Supp}(V))$, or simply denoted as w_V .

Linear code. An $[n, k]_{q^m}$ linear code $\mathcal{C}$ is a k -dimensional subspace of $\mathbb{F}_{q^m}^n$. It can be described either by a full-rank generator matrix $\mathbf{G} \in \mathbb{F}_{q^m}^{k \times n}$ or by a full-rank parity-check matrix $\mathbf{H} \in \mathbb{F}_{q^m}^{(n-k) \times n}$, where $\mathbf{G}$ satisfies $\mathcal{C} = \{\mathbf{xG} : \mathbf{x} \in \mathbb{F}_{q^m}^k\}$; $\mathbf{H}$ satisfies $\mathcal{C} = \{\mathbf{x} \in \mathbb{F}_{q^m}^n : \mathbf{Hx}^\top = \mathbf{0}\}$.

Syndrome. The vector $\mathbf{Hv}^\top$ is called the *syndrome* of $\mathbf{v}$ with respect to $\mathbf{H}$.

Minimum distance. The minimum rank distance of a linear code $\mathcal{C}$, denoted by $d_R(\mathcal{C})$, is the minimum rank weight among all non-zero codewords in $\mathcal{C}$. Such a code can uniquely decode up to $\left\lfloor \frac{d_R(\mathcal{C})-1}{2} \right\rfloor$ errors, and is typically denoted by $[n, k, d_R(\mathcal{C})]$.

Gaussian Coefficients. The number of $\mathbb{F}_q$ -subspaces of dimension w in $\mathbb{F}_{q^m}$ is given by the Gaussian coefficient

$$\begin{bmatrix} m \\ w \end{bmatrix}_q = \prod_{i=0}^{w-1} \frac{q^m - q^i}{q^w - q^i} = \Theta(q^{w(m-w)}).$$

Augmented Gabidulin code. Let $\mathcal{G}_{\mathbf{g}}^+(n_1 n_2, n', k, m)$ denote an Augmented Gabidulin code with a generator matrix $\mathbf{G} \in \mathbb{F}_{q^m}^{k \times n_1 n_2}$. Its decoding algorithm is denoted by $\mathcal{D}_{\mathcal{G}_{\mathbf{g}}^+}(\cdot)$, which corrects rank errors up to the designed decoding radius, which corrects up to

$$t \leq \left\lfloor \frac{n' - k + \varepsilon}{2} \right\rfloor$$

rank errors with decoding failure rate

$$q^{\delta(n' - n_1 n_2)} \sum_{i=0}^{\varepsilon-1} \prod_{j=0}^{i-1} \frac{(q^\delta - q^j)(q^{(n_1 n_2 - n')} - q^j)}{q^i - q^j},$$

where

$$\varepsilon \in \{1, 2, \dots, \min(n_1 n_2 - n', n' - k)\}.$$

Fold and Unfold. For a vector $\mathbf{v} \in \mathbb{F}_{q^m}^{n-1n_2}$, the folding operator

$$\text{Fold}(\mathbf{v}) \in \mathbb{F}_{q^m}^{n_2 \times n_1}$$

maps $\mathbf{v}$ into a ring representation, and **Unfold** denotes its inverse.

Structure of error vectors. A pairwise error vector is called blended since all blocks are sampled from partially joint supports and we define the rank weight profile of the error vector as the associate matrix as follows.

For a positive integer ℓ , vectors $\mathbf{t} = (t_1, \dots, t_\ell) \in \mathbb{Z}^\ell$, $\mathbf{n} = (n_1, \dots, n_\ell) \in \mathbb{Z}^\ell$, we define $\mathbf{t} \times \mathbf{n} \stackrel{\text{def}}{=} (t_1 \times n_1, \dots, t_\ell \times n_\ell)$, and $\mathbf{E} = (\mathbf{E}_1, \dots, \mathbf{E}_\ell) \in \prod_{i=1}^\ell \mathbb{F}_{q^m}^{t_i \times n_i} \stackrel{\text{def}}{=} \mathbb{F}_{q^m}^{\ell \times \mathbf{n}}$. We write its associated rank matrix with intersection supports as

$$\mathcal{A}_{\mathbf{E}} = (\dim(\text{Supp}(\mathbf{E}_i) \cap \text{Supp}(\mathbf{E}_j)))_{\substack{1 \leq i \leq t \\ 1 \leq j \leq t}} \in \mathbb{N}^{t \times t}.$$

The finite set $\mathcal{S}_{\mathbf{A}}^{\mathbf{t} \times \mathbf{n}}(\mathbb{F}_{q^m})$. For a given rank matrix $\mathbf{A} \in \mathbb{Z}^{\ell \times \ell}$, $\mathbf{t} \in \mathbb{Z}^\ell$, and $\mathbf{n} \in \mathbb{Z}^\ell$, define

$$\mathcal{S}_{\mathbf{A}}^{\mathbf{t} \times \mathbf{n}}(\mathbb{F}_{q^m}) = \{\mathbf{E} = (\mathbf{E}_1, \dots, \mathbf{E}_\ell) \in \mathbb{F}_{q^m}^{\mathbf{t} \times \mathbf{n}} \mid \mathbf{E}_i \in \mathbb{F}_{q^m}^{t_i \times n_i}, \mathcal{A}_{\mathbf{E}} = \mathbf{A}\}.$$

If $\mathbf{t} = (1, \dots, 1)$, the above definition becomes the vector case, we denote it as $\mathcal{S}_{\mathbf{A}}^{\mathbf{n}}(\mathbb{F}_{q^m})$.

For the given rank matrix $\mathbf{A} \in \mathbb{Z}^{2 \times 2}$ and $\mathbf{n} = (n, n)$ and $\mathbf{n}_1 = (n_1, n_1)$ in the key generation, the error vectors $\mathbf{X}, \mathbf{Y}$ are sampled randomly from the set $\mathcal{S}_{\mathbf{A}}^{\mathbf{n} \times \mathbf{n}_1}(\mathbb{F}_{q^m})$.

Similarly, for the 2×2 given rank matrix $\mathbf{A}'$ and $\mathbf{n}_2 = (n_2, n_2)$, $\mathbf{n}' = (n, n + n_1)$ in the encryption phase, the errors $(\mathbf{R}_1 \mid \mathbf{E} \mid \mathbf{R}_2) \in \mathbb{F}_{q^m}^{\mathbf{n}_2 \times \mathbf{n}}$ are chosen randomly from the set $\mathcal{S}_{\mathbf{A}'}^{\mathbf{n}_2 \times \mathbf{n}'}(\mathbb{F}_{q^m})$. Here, we do not consider $\mathbf{E}$ since $\text{Supp}(\mathbf{E} = \text{Supp}(\mathbf{R}_2))$.

This partially joint-support structure ensures that the blended-block errors satisfies the decoding bound required by the code.

Random oracles and hash functions. We use $\text{shake256}(\cdot)$ as a pseudorandom generator that expands a seed into structured algebraic objects. In the KEM transformation, we further use two hash functions

$$\mathcal{G} : \{0, 1\}^* \rightarrow \{0, 1\}^{512}, \quad \mathcal{K} : \{0, 1\}^* \rightarrow \{0, 1\}^{512},$$

instantiated respectively by SHA3-512 and SHAKE256. The function $\mathcal{ID}(\cdot)$ denotes a deterministic extraction function that derives entropy from the public key.

Key generation seeds. The key generation algorithm uses two independent seeds

$$\rho, \sigma \xleftarrow{\$} \{0, 1\}^\lambda,$$

where λ is the security level, the seed ρ determines the public matrix $\mathbf{h} \in \mathcal{R}^2$, by selecting randomly from $\mathcal{S}_{\mathbf{M}_1}^{3n}(\mathbb{F}_{q^m})$.

Encryption seed. Given $\mathbf{pk} = (\rho, \mathbf{s})$, the encryption algorithm first derives $\mathbf{H}$ and $\mathbf{g}$ in Augmented Gabidulin code from $\text{shake256}(\rho)$, where $\text{shake256}(\cdot)$ denotes the SHAKE256 hash function and ρ is called a seed. In Encryption phase $(\mathbf{R}_1, \mathbf{E}, \mathbf{R}_2)$ is choosed at random from a finite set generated by The Seed3.

This partially joint-support structure ensures that the combined error satisfies the decoding bound required by the Augmented Gabidulin code.

Message and ciphertext spaces. Messages are vectors $\mathbf{m} \in \mathbb{F}_{q^m}^k$. Ciphertexts are pairs $\mathbf{C} = (\mathbf{C}_1, \mathbf{C}_2)$ with $\mathbf{C}_1 \in \mathbb{F}_{q^m}^{n_2 \times n_1}$ and $\mathbf{C}_2 \in \mathbb{F}_{q^m}^{n_2 \times n_1}$. All objects are encoded into byte strings using compact representations.

KEM-specific notation. In the salted variant of the Fujisaki–Okamoto (SFO) transformation, we use randomness

$$(k, \theta) \leftarrow \mathcal{G}(\mathcal{ID}(\mathbf{pk}), \mathbf{m}, \text{salt}),$$

and derive the session key as

$$K := \mathcal{K}(k, \mathbf{ct}).$$

The decapsulation algorithm uses implicit rejection with a fallback value $\xi \in \{0, 1\}^{512}$.

All randomness in the scheme is generated either uniformly at random or via domain-separated hash expansions, and all algebraic operations are performed over $\mathbb{F}_{q^m}$ as specified.

1.3 IND-CPA Secure BAG-Loong.PKE

Building on the notations and symbols established in Section 1.2, we now proceed to construct an IND-CPA secure public-key encryption scheme, which we denote as BAG-Loong.PKE. Its formal description is given in Algorithm 1.

Concretely, we improve the Loong framework [17] by introducing small blended-block errors with partially joint supports during both key generation and encryption. This modification effectively reduces the overall parameter sizes of the scheme. Furthermore, we adopt the augmented Gabidulin (AG) code as the auxiliary error-correcting code. By exploiting the multi-syndrome decoding technique inherent to this AG code, we achieve a significant enhancement in decoding performance. This improvement, in turn, allows us to choose smaller system parameters, which consequently leads to more compact public-key and ciphertext sizes.

BAG-Loong. PKE :

KeyGen(1^λ):

- Take positive integers n, n_1, n_2, m , a prime power $q = 2$, and a small rank weight matrix $\mathbf{A} \in \mathbb{Z}^{2 \times 2}$;
- Sample seeds $\rho_{\mathbf{H}} \xleftarrow{\$} \{0, 1\}^{512}$ and compute the matrix $\mathbf{H} \in \mathbb{F}_{q^m}^{n \times n} := \text{shake256}(\rho_{\mathbf{H}})$.
- Sample error matrices $\mathbf{X}, \mathbf{Y}$ randomly from $\mathcal{S}_{\mathbf{A}}^{n \times n_1}(\mathbb{F}_{q^m})$, where the associated rank weight matrix $\mathcal{A}_{(\mathbf{X}, \mathbf{Y})} = \mathbf{A}$, $\mathbf{n}_2 = (n_2, n_2)$, $\mathbf{n}' = (n_1, n_1)$, $1 \in \text{Supp}(\mathbf{Y})$.
- Compute $\mathbf{S} := \mathbf{H}\mathbf{X} + \mathbf{Y}$.
- Output public key $\text{pk} := (\rho_{\mathbf{H}}, \mathbf{S})$, secret key $\text{sk} := \mathbf{X}$.

Enc(pk, $\mathbf{m} \in \mathbb{F}_{q^m}^k$):

- Set $\mathbf{H} \in \mathbb{F}_{q^m}^{n \times n} := \text{shake256}(\rho_H)$, and let $\mathbf{G}$ be the generator matrix of the Augmented Gabidulin code $\mathcal{G}_g^+(n_1 n_2, n', k, m)$.
- For the given small rank weight matrix $\mathbf{A}'$, sample error matrices $\mathbf{R}_1 \in \mathbb{F}_{q^m}^{n_2 \times n}$, $\mathbf{E} \in \mathbb{F}_{q^m}^{n_2 \times n_1}$, $\mathbf{R}_2 \in \mathbb{F}_{q^m}^{n_2 \times n}$ randomly from $\mathcal{S}_{\mathbf{A}'}^{n_2 \times n_1}(\mathbb{F}_{q^m})$ such that the associated rank weight matrix $\mathcal{A}_{(\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2)} = \mathbf{A}'$, where $\mathbf{n}_1 = (n, n + n_1)$, $\mathbf{E} | \mathbf{R}_2$ denotes the concatenation of $\mathbf{E}$ and $\mathbf{R}_2$.
- Output ciphertext pair $\mathbf{C} = (\mathbf{C}_1, \mathbf{C}_2)$, where $\mathbf{C}_1 := \mathbf{R}_2 \mathbf{H} + \mathbf{R}_1$, $\mathbf{C}_2 := \mathbf{R}_2 \mathbf{S} + \mathbf{E} + \text{Fold}(\mathbf{mG})$.

Dec(sk, $\mathbf{C}$):

- Output plaintext $\mathbf{m} := \mathcal{D}_{\mathcal{G}_g^+}(\text{Unfold}(\mathbf{C}_2 - \mathbf{C}_1 \mathbf{X}))$, where $\mathcal{D}_{\mathcal{G}_g^+}(\cdot)$ denotes the decoding algorithm for the Augmented Gabidulin code $\mathcal{G}_{\mathbf{g}}^+(n_1 n_2, n', k, m)$.

Algorithm 1: BAG-Loong.PKE

Correctness:

For the ciphertexts of BAG-Loong, we have

$$\mathbf{C}_1 := \mathbf{R}_2 \mathbf{H} + \mathbf{R}_1, \quad \mathbf{C}_2 := \mathbf{R}_2 \mathbf{S} + \mathbf{E} + \text{Fold}(\mathbf{mG}).$$

Accordingly, during the decryption procedure, we compute

$$\mathbf{C}_2 - \mathbf{C}_1 \mathbf{X} := \text{Fold}(\mathbf{mG}) + \mathbf{R}_2 \mathbf{Y} + \mathbf{E} - \mathbf{R}_1 \mathbf{X}.$$

If the rank weight satisfies

$$w_R(\mathbf{R}_2 \mathbf{Y} - \mathbf{R}_1 \mathbf{X}) \leq \left\lfloor \frac{n' - k + \varepsilon}{2} \right\rfloor,$$

the message $\mathbf{m}$ can be correctly recovered by the decoding algorithm $\mathcal{D}_{\mathcal{G}_{\mathbf{g}}^+}(\cdot)$ of the augmented Gabidulin code $\mathcal{G}_{\mathbf{g}}^+(n_1 n_2, n', k, m)$. Thus, the algorithm is correct.

1.4 IND-CCA2 Secure BAG-Loong.KEM

Based on the BAG-Loong.PKE scheme presented in the previous section, we now construct an IND-CCA2 secure key encapsulation mechanism. Since the above PKE scheme

employs the augmented Gabidulin code as its auxiliary code and features a negligible decryption failure probability, we can apply the Fujisaki-Okamoto transformation [3, 12, 11] to convert it into an IND-CCA2 secure KEM in the random oracle model. We denote the resulting KEM as BAG-Loong.KEM.

Concretely, we adopt the SFO [PKE, G, H, len_{salt}] transformation algorithm proposed in [11] to upgrade the IND-CPA secure PKE scheme into an IND-CCA secure KEM scheme. In the following, we present the detailed construction of our scheme based on the aforementioned transformation algorithms.

Let $\mathcal{E}$ be an instance of the BAG-Loong public-key encryption scheme. Let $\mathcal{G} : \{0, 1\}^* \rightarrow \{0, 1\}^{512}$ and $\mathcal{K} : \{0, 1\}^* \rightarrow \{0, 1\}^{512}$ be two key expansion functions, which are implemented by SHA3-512 and SHAKE256 respectively. Let $\mathcal{ID}(\cdot)$ denote the public-key entropy extraction function. The algorithm description of BAG-Loong.KEM is illustrated in Algorithm 2.

BAG-Loong. KEM :

KeyGen(1^λ):

- Call $(\text{pk}', \text{sk}') \leftarrow \mathcal{E}.\text{KeyGen}(1^\lambda)$.
- Uniformly sample $\xi \xleftarrow{\$} \{0, 1\}^{512}$.
- Output public key $\text{pk} := \text{pk}'$ and secret key $\text{sk} := (\text{sk}', \text{pk}', \xi)$.

Encap(pk):

- Uniformly sample $\mathbf{m} \in \mathbb{F}_{q^m}^k$, $\text{salt} \in \mathbb{F}_2^{128}$, and compute $(k, \theta) \leftarrow \mathcal{G}(\mathcal{ID}(\text{pk}), \mathbf{m}, \text{salt})$.
- Call $\text{ct} \leftarrow \mathcal{E}.\text{Enc}(\text{pk}, \mathbf{m}, \theta)$, and generate session key $K \leftarrow \mathcal{K}(k, \text{ct})$.
- Output ciphertext ct , salt , and session key K .

Decap(sk = (sk', pk', ξ), ct, salt):

- Call $\mathbf{m}' \leftarrow \mathcal{E}.\text{Dec}(\text{sk}', \text{ct})$.
- Compute $(k', \theta') \leftarrow \mathcal{G}(\mathcal{ID}(\text{pk}'), \mathbf{m}', \text{salt})$.
- Call $\text{ct}' \leftarrow \mathcal{E}.\text{Enc}(\text{pk}', \mathbf{m}', \theta')$.
- If $\text{ct}' = \text{ct}$, compute and output $K' \leftarrow \mathcal{K}(k', \text{ct}')$; otherwise, return $K' \leftarrow \mathcal{K}(\xi, \text{ct}')$ with implicit rejection.

Algorithm 2: BAG-Loong.KEM

1.5 Parameter Selection

We propose several parameter sets for the BAG-Loong scheme, targeting classical security levels of 128, 256, 384, and 512 bits while offering 80, 128, 192, and 256 bits of quantum security, respectively.

The specific parameter selections are shown in Table 1. Note that $\mathbf{E} \parallel \mathbf{R}_2$ denotes the concatenation of matrix $\mathbf{E}$ and $\mathbf{R}_2$ to become an $n_2 \times n_1 + n$ matrix.

Table 1: BAG-Loong parameter sets

instance	n	m	n'	n_1	n_2	ε	k	$\mathcal{A}_{\mathbf{x}, \mathbf{y}}$	$\mathcal{A}_{\mathbf{R}_1, \mathbf{E} \mathbf{R}_2}$
BAG-Loong-128	42	47	46	10	10	33	3	$\begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix}$	$\begin{pmatrix} 5 & 3 \\ 3 & 5 \end{pmatrix}$
BAG-Loong-256	65	67	67	12	13	51	4	$\begin{pmatrix} 5 & 3 \\ 3 & 6 \end{pmatrix}$	$\begin{pmatrix} 6 & 3 \\ 3 & 6 \end{pmatrix}$
BAG-Loong-384	83	83	83	14	15	66	5	$\begin{pmatrix} 6 & 3 \\ 3 & 6 \end{pmatrix}$	$\begin{pmatrix} 7 & 4 \\ 4 & 7 \end{pmatrix}$
BAG-Loong-512	104	97	97	15	15	73	6	$\begin{pmatrix} 7 & 4 \\ 4 & 7 \end{pmatrix}$	$\begin{pmatrix} 7 & 4 \\ 4 & 7 \end{pmatrix}$

Table 2 provides the polynomial of $f(X)$ is the polynomial used to define the finite field $\mathbb{F}_{q^m}$ as $\mathbb{F}_q[X]/\langle f(X) \rangle$.

Table 2: Polynomials considered for BAG-Loong. $f(X)$ is the polynomial used to define $\mathbb{F}_{q^m}$ as $\mathbb{F}_q[X]/\langle f(X) \rangle$.

Instance	$f(X)$
BAG-Loong-128	$X^{47} + X^5 + 1$
BAG-Loong-256	$X^{67} + X^5 + X^2 + X + 1$
BAG-Loong-384	$X^{83} + X^7 + X^4 + X^2 + 1$
BAG-Loong-512	$X^{104} + X^4 + X^3 + X + 1$

In addition, we give presentation of keys and ciphertexts.

The seed1 is used to generate $\mathbf{X}$, $\mathbf{Y}$, the parameter $\mathbf{g}$ in the augmented Gabidulin code $\mathcal{G}_{\mathbf{g}}^+(n_1 n_2, n', k, m)$. The secret key $\mathbf{sk} = (\mathbf{X}, \text{seed1})$. The public key $\mathbf{pk} = (\rho_H, \mathbf{S})$ is denoted as $\mathbf{pk} = (\text{seed2}, \mathbf{S})$, in which $\rho_H = \text{seed2}$ serves to generate matrix $\mathbf{H}$. The ciphertext $\mathbf{C}$ is represented as $(\mathbf{C}_1, \mathbf{C}_2)$. The public key size is $2 * \lambda/8 + \left\lceil \frac{nn_1 m}{8} \right\rceil$ bytes, and the ciphertext size is $\left\lceil \frac{(nn_2 + n_1 n_2)m}{8} \right\rceil$ bytes.

Table 3 we present the pk and ck sizes.

Table 3: BAG-Loong parameter sizes

Instance	pk size	seed size	ct size	pk+ct size	shared key
BAG-Loong-128	2500	16	3055	5555	16
BAG-Loong-256	6597	32	8384	14981	32
BAG-Loong-384	12152	48	15096	27248	48
BAG-Loong-512	19043	64	21644	40687	64

Table 4 gives the performance data are the averages over 1000 random instances.

Table 4: Millions of CPU cycles of BAG-Loong reference implementation.

Instance	KeyGen	Encaps	Decaps
BAG-Loong-128	12.97	13.94	33.39
BAG-Loong-256	42.82	57.93	91.34
BAG-Loong-384	57.01	61.26	155.50
BAG-Loong-512	80.92	88.96	223.28

2 Design Rationale

2.1 Difficult Problems for Code-based Public-key Cryptography

In this section, we present several difficult problems commonly used in code-based cryptosystems. These hardness assumptions form the security foundation of code-based public-key schemes.

Before that, we provide a new error pattern, that is, blended-block errors. Let $\mathbb{Z}$ denote the all integers and $\mathbf{n} = (n_1, \dots, n_\ell) \in \mathbb{N}^\ell$ with $n = n_1 + \dots + n_\ell$.

Definition 2.1 (Blended-block errors (ℓ -blended errors)). *For a positive integer ℓ , a block error $\mathbf{e} = (\mathbf{e}_1, \dots, \mathbf{e}_\ell)$ with $\mathbf{e}_i \in \mathbb{F}_{q^m}^{n_i}$ is called an ℓ -blended error if its associated support-intersection matrix $\mathcal{A}_{\mathbf{e}} = (\dim(\text{Supp}(\mathbf{e}_i) \cap \text{Supp}(\mathbf{e}_j)))_{\substack{1 \leq i \leq \ell \\ 1 \leq j \leq \ell}}$ is need not diagonal, nor need it be a scalar multiple of the identity matrix $\mathbf{I}_\ell$. In other words, the supports of the error blocks are allowed to overlap in an arbitrary pattern.*

Remark 2.1. *We highlight the following cases:*

- *It is obvious that $\mathcal{A}_{\mathbf{e}}$ is symmetric.*
- *When $\mathcal{A}_{\mathbf{e}}$ is diagonal, the error is ℓ -block, that is, any two common supports of any distinct $\mathbf{e}_i$ and $\mathbf{e}_j$ are mutually disjoint. It is the case in [16].*
- *When $\ell = 1$, the error is the one block and is the general case of RSD.*
- *When $\mathcal{A}_{\mathbf{e}} = c\mathbf{I}_\ell$, the case is the same as RQC, ROLLO.*

In the following we generalize the code-based difficult problem with the blended block-wise errors.

Definition 2.2 (ℓ -BRD problem). *Let $\mathbf{A} \in \mathbb{N}^{\ell \times \ell}$ be a matrix associated with a blended-block error. Given a full-rank matrix $\mathbf{G} \in \mathbb{F}_{q^m}^{k \times n}$ and $\mathbf{y} \stackrel{\text{def}}{=} \mathbf{xG} + \mathbf{e}$, where $\mathbf{x} \in \mathbb{F}_{q^m}^k$ is sampled uniformly and $\mathbf{e} \in \mathcal{S}_{\mathbf{A}}^{\mathbf{n}}(\mathbb{F}_{q^m})$, the blended-block rank decoding problem ℓ -BRD($\mathbf{n}, k, \mathbf{A}$) asks to recover $\mathbf{x}$ and $\mathbf{e}$.*

Definition 2.3 (ℓ -BRSD problem). *Let $\mathbf{A} \in \mathbb{N}^{\ell \times \ell}$ be a matrix associated with a blended-block error. Let $\mathbf{H}$ be a parity-check matrix of a random $[n, k]_{q^m}$ linear code $\mathcal{C}$, and let $\mathbf{s} \in \mathbb{F}_{q^m}^{n-k}$. The blended-block rank syndrome decoding problem ℓ -BRSD($\mathbf{n}, k, \mathbf{A}$) asks to find $\mathbf{e} \in \mathcal{S}_{\mathbf{A}}^{\mathbf{n}}(\mathbb{F}_{q^m})$ such that $\mathbf{s} = \mathbf{He}^\top$.*

When $\ell = 1$, The ℓ -BRSD problem becomes the RSD problem, which has been shown to be NP-hard as it can be probabilistically reduced to the syndrome decoding problem in the Hamming metric [10]. It is well known that the latter is NP-hard [6].

By the duality, the hardness of the ℓ -BRD problem and ℓ -BRSD problem are equivalent. Furthermore, two problems are also hard because they still find a small-weight error.

The rank support learning problem [9] can be viewed as a generalization of the RD problem, where N instances share the same generator matrix (or the same parity-check matrix in the RSD setting), and the errors have a common support.

A $k \times n$ matrix $B = (m_{ij})$ over $\mathbb{F}_{q^m}$ is said to be *homogeneous* on a support set E if the $\mathbb{F}_q$ -subspace spanned by its entries m_{ij} is equal to E .

Definition 2.4 (ℓ -BRSL problem). *Given $(\mathbf{H}, \mathbf{H}\mathbf{E}^\top)$ and a rank matrix $\mathbf{A}$, where $\mathbf{H} \in \mathbb{F}_{q^m}^{(n-k) \times n}$ is full-rank and $\mathbf{E} = (\mathbf{E}_1 \mid \cdots \mid \mathbf{E}_\ell) \in \mathbb{F}_{q^m}^{N \times n}$ with is such that for each $i \in \{1, \dots, \ell\}$, the matrix $\mathbf{E}_i \in \mathbb{F}_{q^m}^{N \times n_i}$ is homogeneous on support $\mathcal{V}_i$ with $\mathcal{A}_{\mathbf{E}} = \mathbf{A}$, the blended-block rank support learning problem ℓ -BRSL($\mathbf{n}, k, \mathbf{A}, N$) asks to recover the family of subspaces $(\mathcal{V}_i = \text{Supp}(\mathbf{E}_i))_{i \in \{1, \dots, \ell\}}$.*

The above problems is also referred to as the search version, denoted by ℓ -BRSD $\mathbf{n}, k, \mathbf{A}$. We also have the decisional version of the above problems, denoted by ℓ -DBRSD $\mathbf{n}, k, \mathbf{A}$.

We do not consider the general case. For simplicity, we only consider some special forms for the errors. Specifically, in BAG-Loong, the associated rank matrices of the error vectors in key generation and encryption has the form like $\begin{pmatrix} t_1 & g \\ g & t_2 \end{pmatrix}$.

2.2 Criterion for Parameter Choices

The concrete security of the BAG-Loong scheme relies on the two 2-DBRSL problems, against which two main classes of attacks—namely combinatorial and algebraic attacks—are known (see Section 3 for details). To ensure that all configurations meet the required security margins, our parameter selection carefully considers both attack families. Under the currently best-known cryptanalytic estimates, we propose four parameter sets for BAG-Loong, delivering classical security of at least 128, 256, 384, and 512 bits, and quantum security of 80, 128, 192, and 256 bits, respectively.

In the following we consider the parameters in BAG-Loong scheme needed to satisfy the detailed conditions:

- m should be prime in order to avoid algebraic attack.
- BAG-Loong uses augmented Gabidulin code $G_g^+(n_1 n_2, n', k, m)$ as an auxiliary code, which is deterministic, and is able to correct $\leq \left\lfloor \frac{n' - k + \varepsilon}{2} \right\rfloor$ rank errors at least with decoding failure rate

$$\text{DFR} = q^{\delta(n' - n_1 n_2)} \sum_{i=0}^{\varepsilon-1} \prod_{j=0}^{i-1} \frac{(q^\delta - q^j)(q^{(n_1 n_2 - n')} - q^j)}{q^j - q^j}$$

where $\varepsilon \in \{1, 2, \dots, \min(n_1 n_2 - n', n' - k)\}$.

- For the augmented Gabidulin code $G_g^+(n_1 n_2, n', k, m)$, i.e., $n' \leq m$ is a positive integer, and n_1, n_2 are positive integers, satisfying $0 < k < n'$, where k is the dimension of the augmented Gabidulin code and so $2^{km} \geq 2^\lambda$. It must satisfy the decoding capability requirement of the augmented Gabidulin code:

$$w_{\mathbf{R}_2 \mathbf{Y} - \mathbf{R}_1 \mathbf{X}} \leq \left\lfloor \frac{n' - k + \varepsilon}{2} \right\rfloor.$$

- The security of the public-key generation phase $\mathbf{S} = \mathbf{X}\mathbf{H} + \mathbf{Y}$ can be reduced to the 2-BRSL($(n, n), \mathcal{A}_{(\mathbf{X}, \mathbf{Y})}, n_1$) difficult problem, i.e.,

$$(\mathbf{I} \ \mathbf{H}) \begin{pmatrix} \mathbf{Y} \\ \mathbf{X} \end{pmatrix} = \mathbf{S}, \quad (1)$$

and 2-BRSL($(n, n + n_1), \mathcal{A}_{(\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2)}, n_2$)

$$\begin{pmatrix} \mathbf{I}_n & & \mathbf{H}^\top \\ & \mathbf{I}_{n_1} & \mathbf{S}^\top \end{pmatrix} \begin{pmatrix} \mathbf{R}_1^\top \\ \mathbf{E}^\top \\ \mathbf{R}_2^\top \end{pmatrix} = \begin{pmatrix} \mathbf{C}_1^\top \\ \mathbf{C}_1^\top \end{pmatrix}. \quad (2)$$

which must satisfy the best know attacks discussed in Section 3 such as OJ attacks, PRR attacks, MM algebraic attacks.

2.3 Augmented Gabidulin Codes and Decoding

The Augmented Gabidulin code is to append a string of zeros at the end of a Gabidulin code; see [7] for details. Its definition and some properties are put in notations and so are omit here.

In [7] gives a decoding method for the Augmented Gabidulin code $\mathcal{G}_g^+(n, n', k, m)$, in which $(n, n', k, m) \in \mathbb{N}^4$ satisfy $k \leq n' \leq m \leq n$ and $\mathbf{g} = (g_1, \dots, g_{n'}) \in \mathbb{F}_{q^m}^{n'}$ satisfy $w_R(\mathbf{g}) = n'$, and let $\bar{\mathbf{g}} \stackrel{\text{def}}{=} (\mathbf{g} | \mathbf{0}_{n-n'}) \in \mathbb{F}_{q^m}^n$. However, it requires solving a linear system with n' equations and $k + 2\delta - \varepsilon$ unknowns over $\mathbb{F}_{q^m}$, which consumes a large amount of storage and is inefficient. Therefore, we give a new fast decoding algorithm below.

Assume that a noisy codeword $\bar{\mathbf{y}} = \bar{\mathbf{c}} + \bar{\mathbf{e}} \in \mathbb{F}_{q^m}^n$ is given, where $\bar{\mathbf{c}} = \mathbf{x}\mathbf{G}$ belongs to $\mathcal{G}_g^+(n, n', k, m)$, and $w_{\mathbf{e}} \leq \delta$. By construction, $\bar{\mathbf{c}} = (* | 0 \dots 0)$, and therefore the last $n - n'$ coordinates of $\bar{\mathbf{y}}$ are exactly the last $n - n'$ entries of $\bar{\mathbf{e}}$.

Let $\mathbf{y}, \mathbf{c}, \mathbf{e}$ denote the vectors generated by the first n' coordinates of $\bar{\mathbf{y}}, \bar{\mathbf{c}}, \bar{\mathbf{e}}$, respectively. Then

$$y_i = f(g_i) + e_i, \quad 1 \leq i \leq n'.$$

Assume that the dimension of the support of the last $n - n'$ entries of the error vector $\bar{\mathbf{e}}$ is ε . Let E_2 denote the vector space generated by these elements, and let V_2 be the unique monic q -polynomial annihilating the elements of E_2 . Then

$$V_2(y_i) = V_2 \circ f(g_i) + V_2(e_i), \quad 1 \leq i \leq n'. \quad (*)$$

Construct a new Gabidulin code $\mathcal{G}(\mathbf{g}, n', \varepsilon + k)$. It is straightforward that this code has minimum distance

$$d' = n' - \varepsilon + k + 1,$$

and therefore it can correct at most $\delta - \varepsilon$ errors.

Lemma 2.1. *The Gabidulin code $\mathcal{G}(\mathbf{g}, n', \varepsilon + k)$ has minimum distance*

$$d' = n' - \varepsilon + k + 1,$$

and can correct at most $\delta - \varepsilon$ errors.

Proof. By the properties of Gabidulin codes, one directly obtains that the minimum distance is $d' = n' - \varepsilon + k + 1$. Moreover,

$$\delta = \left\lfloor \frac{n' - k + \varepsilon}{2} \right\rfloor = \begin{cases} \frac{n' - k + \varepsilon}{2}, & \text{if } n' - k + \varepsilon \text{ is even,} \\ \frac{n' - k + \varepsilon - 1}{2}, & \text{if } n' - k + \varepsilon \text{ is odd.} \end{cases}$$

Hence, we have $\left\lfloor \frac{n' - k + \varepsilon}{2} \right\rfloor = \delta - \varepsilon$. □

In (*), the vector

$$(V_2 \circ f(g_1), V_2 \circ f(g_2), \dots, V_2 \circ f(g_{n'}))$$

can be viewed as a codeword of the Gabidulin code $\mathcal{G}(\mathbf{g}, n', \varepsilon + k)$, while $(V_2(e_1), \dots, V_2(e_{n'}))$ has rank at most $\delta - \varepsilon$. Therefore, by applying the decoding algorithm for the Gabidulin code $\mathcal{G}(\mathbf{g}, n', \varepsilon + k)$ to the received word $(V_2(y_1), \dots, V_2(y_{n'}))$, one obtains the corresponding codeword $(V_2 \circ f(g_1), V_2 \circ f(g_2), \dots, V_2 \circ f(g_{n'}))$, and then f can be recovered by the left Euclidean algorithm.

Theorem 2.1. *The complexity of the decoding algorithm for $\mathcal{G}_{\mathbf{g}}^+(n, n', k, m)$ is $O(n^2)$ operations over $\mathbb{F}_{q^m}$.*

Proof. We give the cost of each steps in decoding procedure: computing the rank weight of $\mathbf{y}$ needs $(n - n') * \varepsilon$ operations in $\mathbb{F}_{q^m}$, finding the vanishing polynomial needs ε^2 operations, acting the first part of $\mathbf{y}$ needs $n' * \varepsilon$, running the decoding algorithm of Gabidulin code needs n'_2 and left dividing vanishing polynomial needs $k * \varepsilon$ operations in $\mathbb{F}_{q^m}$. So the total complexity of decoding the Augmented Gabidulin code is required. □

2.4 Statement of Design

The BAG-Loong scheme is an IND-CCA2 secure key encapsulation Mechanism (KEM) with no algebraic structure under rank metric, which adopts the design framework of the Loong scheme [17].

The original Loong scheme achieves IND-CPA security based on the general RSL hard problem. As an early unstructured scheme based on rank distance, the parameters chosen in the Loong scheme are vulnerable against improved algebraic attacks [4, 5].

Therefore, larger parameter sizes are required to ensure security. The Loong scheme selects Gabidulin codes as auxiliary codes, thus there is no decoding failure rate. For the errors sampling, the errors are selected from the same support.

BAG-Loong adopts Augmented Gabidulin codes as the multiple syndrome technique to improve the parameter size. We also propose a new effective iterative decoding algorithm for Augmented Gaidulin codes. For noise sampling in key generation (or encryption procedures), we propose a new blended-block errors sampled from mutually partially joint supports in key generation and key encryption, which is totally different from the errors sampling in Loong. In addition, we use Augmented Gabidulin code as multiple syndrome technique to improve to size of parameters.

3 Security Analysis

In this section, we show that the BAG-Loong.PKE scheme is IND-CPA secure under the hardness assumptions of the DBRSL problems.

3.1 Security Analysis in the Classical Model

IND-CPA Security for BAG-Loong.PKE under ROM. We show the IND-CPA security of the scheme presented in Algorithm 1 (BAG-Loong.PKE) under the 2-DBRSL $((n, n), n, \mathcal{A}_{(\mathbf{X}, \mathbf{Y})}, n_1)$ and the 2-DBRSL $((n, n + n_1), n, \mathcal{A}_{(\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2)}, n_2)$ problems. These two problem instances correspond to the forms given in Eq. (1) and Eq. (2), respectively.

Theorem 3.1. *The scheme presented in Algorithm 1 is IND-CPA under the assumption that both the 2-DBRSL $((n, n), n, \mathcal{A}_{(\mathbf{X}, \mathbf{Y})}, n_1)$ and 2-DBRSL $((n, n + n_1), n, \mathcal{A}_{(\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2)}, n_2)$ are hard.*

Proof. We give a concise proof of security via a hybrid argument: starting from the real encryption of $\mathbf{m}_0$, a sequence of games gradually transitions to that of $\mathbf{m}_1$. Each adjacent indistinguishability can be reduced to the 2-DBRSL $((n, n), n, \mathcal{A}_{(\mathbf{X}, \mathbf{Y})}, n_1)$ or 2-DBRSL $((n, n + n_1), n, \mathcal{A}_{(\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2)}, n_2)$ problem.

Game $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^1(\lambda)$ is the real game where the challenge ciphertext is a valid encryption of the message $\mathbf{m}_0$.

Game $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^2(\lambda)$ differs from $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^1(\lambda)$ by replacing $\mathbf{S} = \mathbf{H}\mathbf{X} + \mathbf{Y}$ in the public key $\mathbf{pk}$ with a uniformly random matrix. This corresponds to a 2-DBRSL $((n, n), n, \mathcal{A}_{(\mathbf{X}, \mathbf{Y})}, n_1)$ instance, because $(\mathbf{I} \ \mathbf{H}) \begin{pmatrix} \mathbf{Y} \\ \mathbf{X} \end{pmatrix} = \mathbf{S}$ with $(\mathbf{I} \ \mathbf{H})$ is the parity-check matrix of a random $[2n, n]$ -linear code over $\mathbb{F}_{q^m}$ and $\mathbf{X}, \mathbf{Y}$ are sampled with low associated rank. Any distinguisher between $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^1(\lambda)$ and $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^2(\lambda)$ with advantage ϵ yields a distinguisher for the 2-DBRSL $((n, n), n, \mathcal{A}_{(\mathbf{X}, \mathbf{Y})}, n_1)$ problem with the same advantage. Thus,

$$|\Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^2(\lambda) = 1] - \Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^1(\lambda) = 1]| = \text{Adv}_{((n, n), n, \mathcal{A}_{(\mathbf{X}, \mathbf{Y})}, n_1)}^{2\text{-DBRSL}}(\lambda).$$

Game $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^3(\lambda)$ replaces the ciphertext components $\mathbf{C}'_1 := \mathbf{R}_2\mathbf{H} + \mathbf{R}_1$ and $\mathbf{C}'_2 := \mathbf{R}_2\mathbf{S} + \mathbf{E}$ with uniformly random values. This corresponds to a 2-DBRSL $((n, n + n_1), n, \mathcal{A}_{(\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2)})$

, n_2) instance, as

$$\begin{pmatrix} \mathbf{I}_n & & \mathbf{H}^\top \\ & \mathbf{I}_{n_1} & \mathbf{S}^\top \end{pmatrix} \begin{pmatrix} \mathbf{R}_1^\top \\ \mathbf{E}^\top \\ \mathbf{R}_2^\top \end{pmatrix} = \begin{pmatrix} \mathbf{C}_1^\top \\ \mathbf{C}_2^\top \end{pmatrix},$$

with $\mathbf{H}$ is the parity-check matrix of a random code, $\mathbf{S}$ is the syndrome matrix and $\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2$ are randomly sampled low-rank-weight matrices with the rank weight matrix $\mathcal{A}_{(\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2)}$. Any distinguisher between $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^2(\lambda)$ and $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^3(\lambda)$ with advantage ϵ yields a distinguisher for the 2-DBRSL $((n, n + n_1), n, \mathcal{A}_{(\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2)}, n_2)$ problem with the same advantage. We have then,

$$|\Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^3(\lambda) = 1] - \Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^2(\lambda) = 1]| = \text{Adv}_{((n, n + n_1), n, \mathcal{A}_{(\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2)}, n_2)}^{2\text{-DBRSL}}(\lambda).$$

Game $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^4(\lambda)$ encrypts $\mathbf{m}_1$ instead of $\mathbf{m}_0$ using freshly sampled randomness. The output distributions of $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^3(\lambda)$ and $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^4(\lambda)$ are identical: for any $(\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2)$ in $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^3(\lambda)$ producing $(\mathbf{C}_1, \mathbf{C}_2)$, the mapping $(\mathbf{R}'_1, \mathbf{E}', \mathbf{R}'_2)$ (with $\mathbf{R}'_1 = \mathbf{R}_1$, $\mathbf{R}'_2 = \mathbf{R}_2$, and $\mathbf{E}' = \mathbf{E} + \text{Fold}(\mathbf{m}_0 \mathbf{G}) - \text{Fold}(\mathbf{m}_1 \mathbf{G})$) is a bijection to a valid tuple in $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^4(\lambda)$ producing the same ciphertext. Thus,

$$|\Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^4(\lambda) = 1] - \Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^3(\lambda) = 1]| = 0.$$

Games $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^5(\lambda)$ and $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^6(\lambda)$ replicate $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^2(\lambda)$ and $\mathbf{G}_{\mathcal{E}, \mathcal{A}}^1(\lambda)$, respectively, except that the challenge message $\mathbf{m}_1$ is encrypted instead of $\mathbf{m}_0$. By the similar reductions, we have

$$\begin{aligned} |\Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^5(\lambda) = 1] - \Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^4(\lambda) = 1]| &= \text{Adv}_{((n, n + n_1), n, \mathcal{A}_{(\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2)}, n_2)}^{2\text{-DBRSL}}(\lambda), \\ |\Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^6(\lambda) = 1] - \Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^5(\lambda) = 1]| &= \text{Adv}_{((n, n), n, \mathcal{A}_{(\mathbf{X}, \mathbf{Y})}, n_1)}^{2\text{-DBRSL}}(\lambda). \end{aligned}$$

Putting all these together, the advantage of an adversary $\mathcal{A}$ against the IND-CPA experiment is bounded as

$$\begin{aligned} \text{Adv}_{\text{BAG-Loong, PKE}, \mathcal{A}}^{\text{IND-CPA}}(\lambda) &= |\Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^6(\lambda) = 1] - \Pr[\mathbf{G}_{\mathcal{E}, \mathcal{A}}^1(\lambda) = 1]| \\ &\leq 2 \cdot (\text{Adv}_{((n, n), n, \mathcal{A}_{(\mathbf{X}, \mathbf{Y})}, n_1)}^{2\text{-DBRSL}}(\lambda) + \text{Adv}_{((n, n + n_1), n, \mathcal{A}_{(\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2)}, n_2)}^{2\text{-DBRSL}}(\lambda)) \\ &\leq \text{negl}(\lambda) \end{aligned}$$

for negl a negligible function. This completes the proof. $\square$

IND-CCA Security for BAG-Loong.KEM under ROM. The aforementioned BAG-Loong public-key encryption scheme achieves IND-CPA security. Benefiting from the adoption of augmented Gabidulin codes, its decryption failure probability can be bounded below $2^{-\lambda}$. Accordingly, we apply the salted version of the Fujisaki-Okamoto(SFO) transformation to upgrade this IND-CPA secure PKE into an IND-CCA2 secure key encapsulation mechanism, namely BAG-Loong.KEM.

We adopt the SFO [PKE, $\mathbf{G}, \mathbf{H}, \text{len}_{\text{salt}}$] transformation algorithm proposed in [11]. In the Random Oracle (RO) model, this multi-user transformation admits the following security reduction:

Lemma 3.1 (SFO Transformation in RO Model [11]). *Let PKE be a public-key encryption scheme, and let KEM be the KEM constructed as $\text{KEM} := \text{SFO}[\text{PKE}, \text{G}, \text{H}, \text{len}_{\text{salt}}]$. If PKE is $\delta(n_u)$ -correct, then KEM is $\delta(n_u)$ correct in the random oracle model. Let the number of queries to ChALL_j be bounded by n_c and $\mathfrak{L}_{S_j}$ represent the set of salt values sampled by during those queries. Consider a IND_{n_c, n_u} -CPA adversary $\mathcal{B}$ against KEM, issuing at most q_H queries to H , q_G queries to G , and let $q = (q_H + q_G)$. Suppose that no salt appears in any $\mathfrak{L}_{M_j}$ more than t times. There exists an IND_{n_c, n_u} -CPA adversary $\mathcal{A}$ against PKE such that*

$$\text{Adv}_{\text{KEM}}^{\text{IND}_{n_c, n_u}\text{-CPA}}(\mathcal{B}) \leq \frac{n_u n_c (n_c - 1)}{|\mathcal{M}| \cdot 2^{\text{len}_{\text{salt}}}} + \frac{2t(q_H + q_G)}{|\mathcal{M}|} + 2 \cdot \text{Adv}_{\text{PKE}}^{\text{IND}_{n_c, n_u}\text{-CPA}}(\mathcal{A}),$$

where n_u denotes the number of users, n_c denotes the number of challenge ciphertexts, G, H are hash functions, and $|\mathcal{M}|$ represents the plaintext space size of PKE. Furthermore $\mathcal{A}$ and $\mathcal{B}$ have similar run-time.

Apparently, by the above lemma, BAG-Loong.KEM transformed from the IND-CPA secure BAG-Loong scheme is an IND-CCA2 secure key encapsulation mechanism.

3.2 Security Analysis in the Quantum Model

Since classical computers are special cases of quantum computers, the classical security reduction for IND-CPA security of BAG-Loong still holds in the quantum model, and thus no additional reduction proof is provided in this section.

For the $\text{SFO}[\text{PKE}, \text{G}, \text{H}, \text{len}_{\text{salt}}]$ transformation adopted by BAG-Loong, its multi-user security reduction in the Quantum Random Oracle (QRO) model is given as follows:

Lemma 3.2 (SFO Transformation in QRO Model [11]). *Let PKE be a public-key encryption scheme, and let KEM be the KEM constructed as $\text{KEM} := \text{SFO}[\text{PKE}, \text{G}, \text{H}, \text{len}_{\text{salt}}]$. If PKE is $\delta(n_u)$ -correct, then KEM is $\delta(n_u)$ correct in the random oracle model. Let the number of queries to CHALL_j at index $j \in [n_u]$ be bounded by n_c and let $\mathfrak{L}_{S_j}$ represent the set of salt values sampled during those queries. Consider a IND_{n_c, n_u} -CPA adversary $\mathcal{B}$ against KEM, issuing at most q_H and q_G many queries in total to H and G respectively, with a total depth of d . Suppose that no salt appears in any $\mathfrak{L}_{M_j}$ more than t times.*

Furthermore, assume that $\mathcal{B}$ does not query eCO.Ext on any of its challenge ciphertexts. Then there exists a quantum IND_{n_c, n_u} -CPA adversary $\mathcal{A}$ against PKE such that

$$\begin{aligned} \text{Adv}_{\text{KEM}}^{\text{IND}_{n_c, n_u}\text{-CPA}}(\mathcal{B}) &\leq 4\sqrt{d \cdot \text{Adv}_{\text{PKE}}^{\text{IND}_{n_c, n_u}\text{-CPA}} + \frac{n_u n_c (n_c - 1)}{|\mathcal{M}| \cdot 2^{\text{len}_{\text{salt}}}}} \\ &\quad + 4\sqrt{\frac{dt(q_H + q_G)}{|\mathcal{M}|}} \end{aligned}$$

where n_u denotes the number of users, n_c denotes the number of challenge ciphertexts, G, H are hash functions, and $|\mathcal{M}|$ represents the plaintext space size of PKE.

3.3 Known Attacks

For our scheme, the most effective attacks are combinatorial attacks and algebraic attacks on BRSL problem. About combinatorial attacks, we consider OJ attack [14, 16] and PRR attacks [15] under the blended errors. We will adapt these attacks into the case of blended errors.

In addition, since the numbers of syndromes in 2-DBRSL $((n, n), n, \mathcal{A}_{(\mathbf{X}, \mathbf{Y})}, n_1)$ problem and the 2-DBRSL $((n, n + n_1), n, \mathcal{A}_{(\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2)}, n_2)$ problem are n_1 and n_2 , respectively, we only consider the security of 2-DBRSD $((n, n), n, \mathcal{A}_{(\mathbf{X}, \mathbf{Y})})$ problem and the 2-DBRSD $((n, n + n_1), n, \mathcal{A}_{(\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2)})$ problem since the numbers are small [8].

The above attacks are made on the following model.

Model 1. Let $\beta_1, \dots, \beta_m$ be a basis of $\mathbb{F}_{q^m}$ over $\mathbb{F}_q$. In the combinatorial attack, the rank decoding problem ℓ -BRD $((d_1, d_2), k, \mathbf{A})$ for a code $\mathcal{C}$ of length $n = d_1 + d_2$, dimension k , over $\mathbb{F}_{q^m}$ is reduced to finding a codeword of rank weight $t = a_{1,1} + a_{2,2}$ with a $\mathbf{y} \in \mathbb{F}_{q^m}^n$ at distance t from $\mathcal{C}$ and look for $\mathbf{c} \in \mathcal{C}$ and $\mathbf{e} = (\mathbf{e}_1, \mathbf{e}_2)$ such that $\mathbf{y} = \mathbf{c} + \mathbf{e}$ with $\mathcal{A}_{\mathbf{e}} = \mathbf{A}$. In the code $\mathcal{C}_y = \mathcal{C} \oplus \langle \mathbf{y} \rangle_{\mathbb{F}_{q^m}}$. The error vector satisfies the equation

$$\mathbf{e} \mathbf{H}_y^T = \mathbf{0},$$

where $\mathbf{H}_y \in \mathbb{F}_{q^m}^{(n-k-1) \times n}$ is a systematic parity-check matrix for $\mathcal{C}_y$. We write the coefficient matrix of $\mathbf{e}$ under the basis β into the product $\mathbf{S}\mathbf{C}$, where $\mathbf{S} \in \mathbb{F}_{q^m}^{m \times t}$ and $\mathbf{C} \in \mathbb{F}_q^{t \times n}$ are the support and coefficient matrices respectively.

3.3.1 OJ Combinatorial Attacks

In the following we give the computational complexity of the OJ combinatorial attacks against the 2-DBRSL $((n, n), n, \mathcal{A}_{(\mathbf{X}, \mathbf{Y})}, n_1)$ problem and the 2-DBRSL $((n, n + n_1), n, \mathcal{A}_{(\mathbf{R}_1, \mathbf{E} | \mathbf{R}_2)}, n_2)$ problem.

Theorem 3.2. For the 2-BRSD $(\mathbf{n}, k, \mathbf{A})$ problem with $[\mathbf{n} = (d_1, d_2), k = d_1]$ over $\mathbb{F}_{q^m}$, the associated rank weight matrix A is $\begin{pmatrix} t_1 & g \\ g & t_2 \end{pmatrix}$, and $t = t_1 + t_2 - g$, the complexity of OJ attack is

$$\mathcal{O}((m(t-1) + (d_1 - t_1))^\omega q^{(d_1 - t_1)(t_1 - 1) + t_2}).$$

Proof. We write the coefficient matrix of the error vector under the basis β into the product $\mathbf{S}\mathbf{C}$, where $\mathbf{S}$ and $\mathbf{C}$ are the support and coefficient matrices in the following form respectively. under the given blended-block errors,

$$\mathbf{S} = \left(\begin{array}{c|c} 1 & S' \end{array} \right), \quad \mathbf{C} = \left(\begin{array}{ccc|cc} \mathbf{I}_{t'_1} & \mathbf{0}_{t'_1 \times g} & \mathbf{D}_1 & \mathbf{0}_{t'_1 \times t'_2} & \mathbf{0}_{t'_1 \times d_2 - t'_2} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I}_{t'_2} & \mathbf{D}_2 \\ \hline \mathbf{0}_{g \times t'_1} & \mathbf{I}_g & \mathbf{C}'_1 & \mathbf{0} & \mathbf{C}'_2 \end{array} \right),$$

where $t'_i = t_i - g$. and $S' \in \mathbb{F}_q^{m \times (t-1)}$, $C'_1 \in \mathbb{F}_q^{g \times (d_1 - t_1)}$, and $C'_2 \in \mathbb{F}_q^{g \times (d_2 - t'_2)}$.

Let $\mathbf{S} = \left(\begin{array}{c|c} 1 & \mathbf{s}' \\ \hline \mathbf{0}_{(m-1) \times 1} & \mathbf{S}'' \end{array} \right) \in \mathbb{F}_q^{m \times t}$ where $\mathbf{s}' = S_{\{1\}, \{2..t\}} \in \mathbb{F}_q^{1 \times (t-1)}$ and $\mathbf{S}'' = \mathbf{S}_{\{2..m\}, \{2..t\}} \in \mathbb{F}_q^{(m-1) \times (t-1)}$.

Let $k = d_1$. Splitting C into $\mathbf{B}_1 = \left(\begin{array}{c|c} 1 & \mathbf{c}^* \\ \hline \mathbf{0}_{(t-1) \times 1} & \mathbf{D}_1^* \end{array} \right) \in \mathbb{F}_q^{t \times (k+1)}$ and $\mathbf{B}_2 = \left(\begin{array}{c} \mathbf{0}_{1 \times (d_2-1)} \\ \mathbf{C}_2^* \end{array} \right) \in \mathbb{F}_q^{t \times (d_2-1)}$, where $\mathbf{c}^* = C_{\{1\}, \{2..d_1+1\}} \in \mathbb{F}_q^{d_1+1}$, $\mathbf{C}_1^* = C_{\{2..t\}, \{2..d_1+1\}} \in \mathbb{F}_q^{(t-1) \times k+1}$. For $\mathbf{B}_2$, let $\mathbf{b}_j$ be the j -th column of $\mathbf{B}_2$ and $\mathbf{c}_j^*$ be the j th column of $\mathbf{C}_2^*$ for $j \in \{1..d_2-1\}$.

Then

$$(\mathbf{B}_1 \ \mathbf{b}_j) = \left(\begin{array}{c|c|c} 1 & \mathbf{c}^* & 0 \\ \hline \mathbf{0}_{(t-1) \times 1} & \mathbf{C}_1^* & \mathbf{c}_2^*(j) \end{array} \right)$$

and

$$\begin{aligned} S(\mathbf{B}_1 \ \mathbf{b}_j)T_j &= \left(\begin{array}{c|c} 1 & \mathbf{s}' \\ \hline \mathbf{0}_{(m-1) \times 1} & \mathbf{S}'' \end{array} \right) * \left(\begin{array}{c|c|c} 1 & \mathbf{c}^* & 0 \\ \hline \mathbf{0}_{(t-1) \times 1} & \mathbf{C}_1^* & \mathbf{c}_2^*(j) \end{array} \right) * T_j \\ &= \left(\begin{array}{c|c|c} 1 & \mathbf{c}^* + \mathbf{s}'\mathbf{C}_1^* & \mathbf{s}'\mathbf{c}_2^*(j) \\ \hline \mathbf{0}_{(t-1) \times 1} & \mathbf{S}''\mathbf{C}_1^* & \mathbf{S}''\mathbf{c}_2^*(j) \end{array} \right) * T_j = \mathbf{0}_{m \times m}. \end{aligned}$$

The above equation gives a quadratic multivariate system with m^2 quadratic polynomials and at most $m(t-1) + (d_1 - t_1)t_1 + t_2$ variables.

By first guess the entries of $\mathbf{D}_1^*$ and $\mathbf{c}_2^*(j)$, we get a linear equation system $\mathbf{S}(\mathbf{B}_1 \ \mathbf{b}_j)\mathbf{T}_j = \mathbf{0}$ about the entries of $\mathbf{S}$ with m^2 equations and $m(t-1) + d_1 - t_1$ unknowns. Since there are at most $q^{(n-t_1)(t_1-1)+t_2}$ numbers of $\mathbf{D}_1^*$ and $\mathbf{c}_2^*(j)$ and so the total complexity is required. $\square$

Therefore we obtain the quantum complexity of OJ attack.

Theorem 3.3. For the 2-BRSD($\mathbf{n}, k, \mathcal{A}$) problem with $[\mathbf{n} = (n_1, n_2), k = n_1]$ over $\mathbb{F}_{q^m}$, the associated rank weight matrix is $\begin{pmatrix} t_1 & g \\ g & t_2 \end{pmatrix}$, and $t = t_1 + t_2 - g$, the quantum complexity of OJ attack is

$$\mathcal{O}((m(t-1) + (n_1 - t_1))^\omega q^{((n_1-t_1)(t_1-1)+t_2)/2}).$$

Note that the complexty is related with the order of $\mathbf{n}$ and so we just run all the possiblity and choose the minimum.

3.3.2 PRR Attacks

In [15], authors introduced a sum-rank metric and presented a kind of attack on the SD problem under the sum-rank weight, which can be used to attack IRSD with disjoint errors. For the partially joint supports in BAG-Loong, we also adapt the simliar attack on BRSD problems.

First we consider the 2-BRSD($\mathbf{n}, k, \mathbf{A}$) problem over $\mathbb{F}_{q^m}$ where the given associated rank matrix $\mathbf{A} = \begin{pmatrix} t_1 & g \\ g & t_2 \end{pmatrix}$, with $\mathbf{n} = (d_1 = n, d_2 = n + n_1)$. In fact, the code $\mathcal{C}_{\mathbf{y}}$ over $\mathbb{F}_{2^m}$ in Model 1 is $[d_1 + d_2 = 2n + n_1, k = d_1 + 1]$ linear code and $d_1 + d_2 - k - 1 = n + n_1 - 1$.

We follow the framework of the PRR generic decoder [15], which is a Las Vegas-type randomized algorithm consisting of two main steps:

1. Randomly choose a super-support $F_i \supseteq \text{Supp}(\mathbf{e}_i)$ of dimension $r_i \geq t_i$ for each block.
2. If the chosen super-supports contain the true error supports, recover the error via erasure decoding.

The expected complexity is the product of the cost of one iteration and the expected number of iterations.

Case 1: Column Support Strategy ($d_1, d_2 \geq m$)

When $d_1, d_2 \geq m$, the attack uses column supports, where all supports lie in the common ambient space $\mathbb{F}_2^m$. The attacker chooses:

- a common subspace $\mathcal{G} \subseteq \mathbb{F}_2^m$ of dimension g ,
- two private subspaces $P_i \subseteq \mathbb{F}_2^m$ of dimension $r_i - g$, with $P_i \cap \mathcal{G} = \{0\}$, yielding a total super-support $F = \mathcal{G} + P_1 + P_2$ of dimension $r = r_1 + r_2 - g$.

The probability that a randomly chosen r -dimensional subspace F contains the true error support $E = \text{Supp}(\mathbf{e}_1) + \text{Supp}(\mathbf{e}_2)$ of dimension $t = t_1 + t_2 - g$ is

$$\Pr(\text{success}) = \frac{\binom{m-t}{r-t}_2}{\binom{m}{r}_2}.$$

Using the Gaussian binomial estimates $\binom{m}{r}_2 \approx 2^{r(m-r)}$, this probability is asymptotically

$$\Pr(\text{success}) \approx 2^{-t_1(m-r_1) - t_2(m-r_2) + m}.$$

Hence, the expected number of iterations is

$$\mathbb{E}[\#\text{iterations}] = \Pr(\text{success})^{-1} \approx 2^{t_1(m-r_1) + t_2(m-r_2) - m}.$$

For erasure decoding to yield a unique solution, the system must have at least as many equations as unknowns. The number of equations is $m(d_1 + d_2 - k - 1)$, and the number of unknowns is $d_1(r_1 - g) + d_2(r_2 - g)$. Thus we require

$$d_1(r_1 - g) + d_2(r_2 - g) \leq m(n + n_1 - 1).$$

Additionally,

$$r_1 + r_2 - g \leq m.$$

The cost of one iteration is dominated by Gaussian elimination on a system of size $m(n-k)$, which costs $\mathcal{O}((m(n-k-1))^\omega)$ operations, where $2 \leq \omega \leq 3$ is the linear algebra constant. Therefore, the total complexity of the PRR attack in the column support case is

$$\mathcal{O}\left((m(n+n_1-1))^\omega \cdot 2^{t_1(m-r_1)+t_2(m-r_2)-m}\right)$$

subject to the constraints

$$\begin{cases} t_i \leq r_i \leq m, \\ r_1 + r_2 - g \leq m, \\ d_1(r_1 - g) + d_2(r_2 - g) \leq m(d_1 + d_2 - k - 1), \end{cases} \quad i = 1, 2,$$

and where (r_1, r_2) are chosen to minimize the exponent $t_1(m-r_1) + t_2(m-r_2) - m$.

Case 2: Row Support Strategy ($d_1, d_2 \leq m$)

When $d_1, d_2 \leq m$, the attack uses row supports, where each block's support lies in its own space $\mathbb{F}_2^{n_i}$. In this setting, the private subspaces are chosen independently in each block. The total support size is constrained by the dimension of the nullspace of the parity-check matrix:

$$(r_1 - g) + (r_2 - g) \leq n + n_1 - 1.$$

The success probability is now

$$\Pr(\text{success}) \approx 2^{-t_1(n_1-r_1)-t_2(n_2-r_2)},$$

where the factor $-m$ from the column-support case disappears because the supports are chosen in different spaces and there is no global $\binom{m}{r}$ denominator.

The cost of one iteration is $\mathcal{O}((m^2(n+n_1-1))^\omega)$ [15]. Thus, the total complexity in the row support case is

$$\mathcal{O}\left((m^2(n+n_1-1))^\omega \cdot 2^{t_1(d_1-r_1)+t_2(d_2-r_2)}\right)$$

subject to

$$\begin{cases} t_i \leq r_i \leq d_i, \\ r_1 + r_2 - 2g \leq n + n_1 - 1, \\ d_1 r_1 + d_2 r_2 \leq m(n + n_1 - 1), \end{cases} \quad i = 1, 2,$$

with (r_1, r_2) chosen to minimize the exponent $t_1(d_1 - r_1) + t_2(d_2 - r_2)$.

Case 3: Mixed Strategy ($d_1 \leq m < d_2$).

Block 1 uses row support, Block 2 uses column support ($d_1 \leq m < d_2$).

The attacker chooses:

- $F_1 \subseteq \mathbb{F}_2^{d_1}$ with $\dim(F_1) = r_1$,
- $F_2 \subseteq \mathbb{F}_2^m$ with $\dim(F_2) = r_2$, where $F_2 = \mathcal{G} + P_2$ with $\dim(\mathcal{G}) = g$ and $\dim(P_2) = r_2 - g$.

The success probability is

$$\Pr(\text{success}) \approx 2^{-t_1(d_1-r_1)-t_2(m-r_2)+m}.$$

The number of unknowns is $d_1 r_1 + d_2(r_2 - g)$, and the number of equations is $m(d_1 + d_2 - k - 1)$. Thus we require

$$d_1 r_1 + d_2(r_2 - g) \leq m(n_1 + n_2 - k - 1).$$

The total support constraint becomes

$$(r_1 - g) + (r_2 - g) \leq d_1 + d_2 - k - 1,$$

Therefore, the total complexity in this mixed case is

$$\mathcal{O}\left((m(n + n_1 - 1))^\omega \cdot 2^{t_1(d_1-r_1)+t_2(m-r_2)-m}\right)$$

subject to

$$\begin{cases} t_1 \leq r_1 \leq d_1, & t_2 \leq r_2 \leq m, \\ r_1 + r_2 - 2g \leq n + n_1 - 1, \\ d_1 r_1 + d_2(r_2 - g) \leq m(n + n_1 - 1). \end{cases}$$

We summarize the PRR attack on the problem:

Case 1: $n + n_1 \geq m$.

The PRR cost is $\mathcal{O}((m(n + n_1 - 1))^\omega) 2^{\sum_{i=1}^2 t_i(m-r_i)-m}$, where (r_1, r_2) varies subject to

$$\begin{cases} t_i \leq r_i \leq m \text{ for } 1 \leq i \leq 2. \\ r_1 + r_2 - g \leq m \\ n(r_1 - g) + (n + n_1)(r_2 - g) \leq m(n + n_1 - 1) \end{cases} \quad \text{such that } \sum_{i=1}^2 t_i(m - r_i) - m \text{ is minimized.}$$

Case 2: $n \leq m$. The PRR cost is $\mathcal{O}(m^2(n + n_1 - 1)^\omega) 2^{\sum_{i=1}^2 t_i(m-r_i)}$, where (r_1, r_2) varies subject to

$$\begin{cases} t_1 \leq r_1 \leq n, t_2 \leq r_2 \leq n + n_1. \\ (r_1 - g) + (r_2 - g) \leq (n + n_1 - 1) \end{cases} \quad \text{such that } t_1(n - r_1) + t_2(n + n_1 - r_2) \text{ is minimized.}$$

Case 3: $n \leq m < n + n_1$.

The PRR cost is $\mathcal{O}(m^2(n + n_1 - 1)^\omega) 2^{t_1(n-r_1)+t_2(m-r_2)-m}$, where (r_1, r_2) varies subject to

$$\begin{cases} t_1 \leq r_1 \leq n, t_2 \leq r_2 \leq m. \\ (r_1 - g) + (r_2 - g) \leq (n + n_1 - 1). \\ nr_1 + (n + n_1)(r_2 - g) \leq m(n + n_1 - 1) \end{cases} \quad \text{such that } t_1(n - r_1) + t_2(m - r_2) - m \text{ is minimized.}$$

3.3.3 Algebraic Attacks

In this section, we provide the MaxMinors (MM) algebraic attack on the 2-BRSD problem.

We adopt the main idea in [2] and use the following lemma [5].

Lemma 3.3 (Proposition 2 in [5]). *Let $J \subset \{1, \dots, N - k - 1\}$ with $\#J = t$. Then*

$$P_J = c_{J+k+1} + \sum_{\substack{T_- \subset \{1, \dots, k+1\}, T_+ \subset J+k+1 \\ T = T_- \cup T_+, \#T = t, T_- \neq \emptyset}} c_T(\mathbf{H}_y)_{J,T}.$$

In the following we consider the MM algebraic attack against the 2-BRSD($\mathbf{n} = (n, n + n_1)$, $k = n$, $\mathbf{A}$)-BRSD problem with the associated rank matrix

$$\mathbf{A} = \begin{pmatrix} t_1 & g \\ g & t_2 \end{pmatrix}.$$

where $t = t_1 + t_2 - g$.

Following the approach in [2], we define the following quantities for the MM attack.

Number of Variables ($T_2(\mathbf{n}, \mathbf{A})$) : The number of possible choices for the error support columns.

2. **Number of Equations** ($N_2(\mathbf{n}, \mathbf{A})$): The number of linearly independent MaxMinors equations.

To select a non-zero maximal minor, we choose columns from both blocks. The key insight is that the common support of dimension g must be distributed between the two blocks.

For block 1 (length n), we choose $i \geq t_1 - g$ columns at least, but not exceed t_1 columns. Thus, we choose $t - i$ columns in the block 2.

- Choose $t_1 - g$ private columns from the remaining $n - i_1$ columns: $\binom{n-i_1}{t_1-g}$

Therefore:

$$T_2(\mathbf{n}, \mathbf{A}) = \sum_{i=t_1-g}^{t_1} \binom{n}{i} \binom{n+n_1}{t-i}$$

Number of Equations $N_2(\mathbf{n}, \mathbf{A})$. According to Lemma 3.1, the MaxMinors equations are constructed from subsets

$$J \subset \{1, 2, \dots, 2n + n_1 - n - 1\}$$

with $|J| = t$.

Thus, the number of equations is the number of ways to choose t elements from $n + n_1 - 1$ positions:

$$(N_2(\mathbf{n}, \mathbf{A}) = \binom{n + n_1 - 1}{r})$$

Therefore, we summarize the Complexity of the MaxMinors Attack.

Theorem 3.4. *For the mixed block code $[\mathbf{n} = (n + n_1, n), k = n]$ over $\mathbb{F}_{q^m}$, the associated rank matrix is $\mathbf{A} = \begin{pmatrix} t_1 & g \\ g & t_2 \end{pmatrix}$ with $t = t_1 + t_2 - g$. Define $N_2(\mathbf{n}, \mathbf{A}) = \binom{n + n_1 - 1}{t}$ and*

$$T_2(\mathbf{n}, \mathbf{A}) = \sum_{i=t_1-g}^{t_1} \binom{n}{i} \binom{n + n_1}{t-i}.$$

If $mN_2(\mathbf{n}, \mathbf{A}) \geq T_2(\mathbf{n}, \mathbf{A}) - 1$, then the cost of MM attack to the hard problem $[2n + n_1, n]$ with 2-hybrid-error-blocks $\mathbf{A}$ is

$$\mathcal{O}(mN_2(\mathbf{n}, \mathbf{A})(T_2(\mathbf{n}, \mathbf{A}))^{\omega-1}),$$

otherwise, the cost of hybrid approach is

$$\mathcal{O}\left(\min_{\substack{\mathbf{a} = (a_1, a_2) \\ mN_2(\mathbf{n}, \mathbf{A}) \geq T_2(\mathbf{n} - \mathbf{a}, \mathbf{A}) - 1}} q^{(\mathbf{a} \cdot \mathbf{r})mN_2(\mathbf{n}, \mathbf{A})(T_2(\mathbf{n} - \mathbf{a}, \mathbf{A}))^{\omega-1}}\right).$$

If $n_1 = 0$, the above result is also for the 2-BRSD($\mathbf{n} = (n, n + n_1)$, $k = n$, $\mathbf{A}$)-BRSD problem.

3.4 Estimates on Concrete Security Strength

In this subsection we give the concrete bit complexity of the above attacks on (3, 1)-BIRSD and (4, 2)-BIRSD problems in Table 5, in which a and b denote the attacks complexity on the first problem and the second problem in (a, b) , respectively.

Table 5: Bit complexity of known attacks on BAG-Loong

Instance	MM attack	PRR attack	PRR quantum	O-J attack	OJ quantum	DFR
BAG-Loong-128	(158, 139)	(221, 160)	(134, 104)	(176, 176)	(99, 100)	2^{-130}
BAG-Piglet-256	(286, 292)	(351, 303)	(201, 168)	(271, 327)	(148, 176)	2^{-271}
BAG-Piglet-384	(403, 469)	(433, 447)	(234, 251)	(418, 490)	(222, 259)	2^{-432}
BAG-Piglet-512	(626, 534)	(591, 592)	(314, 315)	(617, 617)	(322, 322)	2^{-558}

4 Performance Analysis

This section provides a reference implementation of the BAG-Loong public-key encryption algorithm based on Augmented Gabidulin codes, including four parameter sets that respectively target 128, 256, 384, and 512 bits of classical security.

The algorithm is built upon the rank-metric cryptosystem, and its core operations are arithmetic operations over the finite field $\text{GF}(2^m)$. The concrete runtime performance is summarized in Table ?? below.

To accommodate different engineering development and deployment environments, we detail the methods, techniques, and practical considerations adopted in our implementation. These are organized into two main aspects: platform compatibility and dependency management, and programming implementation techniques.

1. Platform Compatibility and Dependency Environment

This implementation is written in C++ and is compatible with most Linux development environments. It uses a standard Makefile for compilation and project management. The following external libraries are required:

- **NTL**: Provides support for big integers and finite field arithmetic. Version 11.x or higher is recommended.
- **GMP**: Offers low-level big integer operations and serves as a dependency of NTL.
- **OpenSSL**: Provides SHAKE hash function encapsulation, which is used for hash computations and randomness expansion (e.g., for seed-based key reconstruction).

We recommend that all dependencies be correctly installed prior to compilation. On common Linux distributions, these can be readily installed via package managers. For instance, on Ubuntu systems, one may run:

```
sudo apt install libntl-dev libgmp-dev libssl-dev
```

2. Programming Implementation Techniques

We now describe the key programming techniques employed in our implementation.

First, we adopt a modular design that separates distinct functionalities—including finite field operations, Augmented Gabidulin encoding and decoding, q -polynomial handling, key and ciphertext parsing, and hash/randomness encapsulation—into independent modules. This architectural choice greatly facilitates future maintenance and component replacement.

Second, regarding the finite field interface (denoted as FFI), the primary interface is provided by the header files `ffi_field.h`, `ffi_elt.h`, and `ffi_vec.h`, which together implement arithmetic operations over $\text{GF}(2^m)$. The current implementation relies on the NTL library; however, as long as the defined interface specifications are strictly followed, the underlying backend can be seamlessly replaced with alternative high-performance libraries (e.g., MPFQ).

Third, we optimize key and ciphertext representation by employing a seed-expansion approach to compress key sizes. Specifically, the public key is represented as (seed_1, s) , and the private key as (seed_2, x) . Using SHAKE256, the full key structure can be reconstructed on demand, which effectively reduces communication and storage overhead.

Finally, the performance results obtained from our implementation is reported in Table 4. All data presented in the table are averages computed over 1000 independent random instances.

5 The Features of BAG-Loong

Code-based public-key cryptography has become one of the main directions in post-quantum cryptography. In particular, rank-metric codes provide an alternative to the Hamming metric and have attracted significant attention due to their relatively compact public key sizes, which can be comparable to those of lattice-based schemes. Representative proposals include ROLLO (a merger of LAKE, LOCKER and Ouroboros-R) and RQC [13], which were selected in the second round of the NIST PQC standardization process. The first code-based public key encryption scheme with no structure is Loong scheme [17].

However, algebraic attacks against rank-metric code-based schemes [4, 5] have shown that several constructions fail to achieve their claimed security levels as well as Loong even if there is so-called no algebraic structure. To maintain the desired security strength, parameter sizes must typically be increased significantly, which prevents these schemes from advancing to the third round of the NIST competition.

As a consequence, NIST has explicitly encouraged further research on rank-metric-based cryptosystems. Recently, techniques such as multiple-syndrome decoding [1, 7, 2] and disjoint-block noise sampling [16] have been proposed to reduce parameter sizes significantly.

Below we highlight the key features of BAG-Loong scheme, followed by a comparative parameter size analysis against the above unstructured code-based schemes.

5.1 The main Advantages of BAG-Loong

BAG-Loong is an IND-CCA2 secure KEM that improves upon the original Loong scheme [17] by incorporating blended-block errors and the multiple-syndrome technique based on Augmented Gabidulin codes. The main distinguishing features of our scheme are as follows:

- We employ blended-block errors sampled from mutually partially joint supports during both key generation and encryption, which enables more compact and flexible parameter configurations;
- The public key is free of any algebraic structure, offering a conservative and long-term security guarantee;
- We propose a new effective iterative decoding algorithm for Augmented Gabidulin codes, which significantly improves the decoding performance;
- The scheme achieves IND-CCA2 security in the random oracle model via the salted variant of the Fujisaki–Okamoto (SFO) transformation.

5.2 Comparison with Other Unstructured Cryptosystems.

To evaluate the performance and security trade-offs of our proposed BAG-Loong scheme, we compare it with three representative unstructured rank-based cryptosystems, namely Loong [17], Multi-UR-AG [7], and LRPC-MS [1] schemes.

The original Loong scheme achieves IND-CPA security under the general RSL hard problem and is instantiated to provide 128-bit and 256-bit classical security levels. However, as an early unstructured rank-based design, its parameter choices are vulnerable to improved algebraic attacks. Consequently, to maintain the claimed security, Loong requires relatively large parameters, which in turn results in considerably large public-key and ciphertext sizes. Notably, Loong employs Gabidulin codes as auxiliary codes, ensuring a zero decryption failure rate.

The Multi-UR-AG scheme, proposed in [7], is likewise unstructured and achieves IND-CPA security under the RSL problem. It adopts the algorithmic framework of Loong but replaces the auxiliary Gabidulin codes with Augmented Gabidulin codes. While this modification reduces the parameter sizes compared to Loong, Multi-UR-AG does not employ the block-wise noise technique.

The unstructured KEM based on LRPC codes, called LRPC-MS, introduced in [1], also relies on the RSL problem as its underlying hardness assumption. By leveraging the multiple-syndrome method, this mechanism achieves notable improvements in decoding efficiency. Nevertheless, similar to Multi-UR-AG, this scheme does not adopt the block-wise noise technique.

Table 6: Parameter comparison with other schemes.

Scheme	pk size	ck size	pk+ck size (expressed in KB)
Multi-UR-AG-128 [7]	4.1	6.9	11.0
Loong-128 [17]	10.9	16.0	26.9
LRPC-MS-128[1]	4.1	3.1	7.2
BAG-Loong-128	2.5	3.1	5.6

References

- [1] Aguilar-Melchor, C., Aragon, N., Dyseryn, V., Gaborit, P., Zémor, G.: Lrpc codes with multiple syndromes: near ideal-size kems without ideals. In: International Conference on Post-Quantum Cryptography. pp. 45–68. Springer (2022)
- [2] Aragon, N., Briaud, P., Dyseryn, V., Gaborit, P., Vinçotte, A.: The blockwise rank syndrome learning problem and its applications to cryptography. In: International Conference on Post-Quantum Cryptography. pp. 75–106. Springer (2024)
- [3] Aragon, N., Gaborit, P., Hauteville, A., Tillich, J.P.: A new algorithm for solving the rank syndrome decoding problem. In: 2018 IEEE International Symposium on Information Theory (ISIT). pp. 2421–2425. IEEE (2018)

- [4] Bardet, M., Briaud, P., Bros, M., Gaborit, P., Neiger, V., Ruatta, O., Tillich, J.: An algebraic attack on rank metric code-based cryptosystems. In: EUROCRYPT 2020. pp. 64–93. Springer (2020)
- [5] Bardet, M., Bros, M., Cabarcas, D., Gaborit, P., Perlner, R.A., Smith-Tone, D., Tillich, J., Verbel, J.A.: Improvements of algebraic attacks for solving the rank decoding and minrank problems. In: ASIACRYPT 2020. pp. 507–536. Springer (2020)
- [6] Berlekamp, E., McEliece, R., Van Tilborg, H.: On the inherent intractability of certain coding problems (corresp.). IEEE Transactions on Information theory **24**(3), 384–386 (1978)
- [7] Bidoux, L., Briaud, P., Bros, M., Gaborit, P.: RQC revisited and more cryptanalysis for rank-based cryptography. IEEE Trans. Inf. Theory **70**(3), 2271–2286 (2024)
- [8] Debris-Alazard, T., Tillich, J.P.: Two attacks on rank metric code-based schemes: Ranksign and an ibe scheme. In: ASIACRYPT 2018. pp. 62–92. Springer (2018)
- [9] Gaborit, P., Hauteville, A., Phan, D.H., Tillich, J.: Identity-based encryption from codes with rank metric. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017. pp. 194–224. Springer (2017)
- [10] Gaborit, P., Zémor, G.: On the hardness of the decoding and the minimum distance problems for rank codes. IEEE Trans. Inf. Theory pp. 7245–7252 (2016)
- [11] Glabush, L., Hövelmanns, K., Stebila, D.: On the multi-target security of post-quantum key encapsulation mechanisms. IACR Commun. Cryptol. **3**(1), 20 (2026)
- [12] Hofheinz, D., Hövelmanns, K., Kiltz, E.: A modular analysis of the fujisaki-okamoto transformation. In: TCC 2017. pp. 341–371. Springer (2017)
- [13] Melchor, C.A., Aragon, N., Bettaieb, S., Bidoux, L., Blazy, O., Deneuville, J.C., Gaborit, P., Zémor, G.: Rank quasi-cyclic (rqc). Second Round submission to NIST Post-Quantum Cryptography call (2020)
- [14] Ourivski, A.V., Johansson, T.: New technique for decoding codes in the rank metric and its cryptography applications. Probl. Inf. Transm. pp. 237–246 (2002)
- [15] Puchinger, S., Renner, J., Rosenkilde, J.: Generic decoding in the sum-rank metric. IEEE Transactions on Information Theory **68**(8), 5075–5097 (2022)
- [16] Song, Y., Zhang, J., Huang, X., Wu, W.: Blockwise rank decoding problem and LRPC codes: Cryptosystems with smaller sizes. In: ASIACRYPT 2023. pp. 284–316. Springer (2023)
- [17] Wang, L.P.: Loong: a new ind-cca-secure code-based kem. In: 2019 IEEE International Symposium on Information Theory (ISIT). pp. 2584–2588. IEEE (2019)